

Worksheet

Rohan Alexander

7 October, 2026

Work in your Assignment II teams. Put every script, the cache folder, and the parquet files in your team repo and commit as you go.

Set-up

1. Make an account at <https://openrouter.ai>. Under Settings, then Keys, create a key. It starts with `sk-or-v1-`.
2. Add ten dollars of credit and, under Settings, set a monthly limit of ten dollars.
3. Make a project and put the key in `.env`.

```
uv init silicon_samples
cd silicon_samples
uv add openai python-dotenv polars numpy
git init
echo ".env" >> .gitignore
echo 'OPENROUTER_API_KEY=sk-or-v1-...' > .env
```

Check:

```
uv run python -c "from dotenv import load_dotenv; import os; load_dotenv(); print('OPENROUTER_API_KEY' in os.environ)"
git status
```

The first line prints `True`. `git status` does not list `.env`.

4. Save the following as `silicon.py`. It is not run on its own and prints nothing. Each later script starts with `from silicon import *`, which loads it, and you run that script with `uv run`, for instance `uv run 00-check.py`.

```

#### Preamble ####
# Purpose: Client, models, schema, and one cached structured call, shared by the worksheet s
# Author: Your names
# Date: 7 October 2026
# Pre-requisites: uv add openai python-dotenv polars numpy; OPENROUTER_API_KEY in .env

import hashlib
import json
import os
from pathlib import Path

from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.environ["OPENROUTER_API_KEY"],
    timeout=60,
    max_retries=2,
)

MODELS = ["deepseek/deepseek-v4-flash-0731", "z-ai/glm-5.3-flash", "meta/muse-glimmer-30b"]
OPTIONS = ["Democratic", "Republican", "Other", "Would not vote"]
SCHEMA = {
    "type": "object",
    "properties": {
        "vote": {"type": "string", "enum": OPTIONS},
        "confidence": {"type": "integer", "minimum": 0, "maximum": 100},
    },
    "required": ["vote", "confidence"],
    "additionalProperties": False,
}

BUDGET_USD = 2.00
CACHE = Path("cache")
CACHE.mkdir(exist_ok=True)
spent = 0.0

def total_spent():
    """Dollars spent so far in this run. Call this rather than reading `spent`, which `import
    return spent

```

```

def ask(model, persona, question, temperature=1.0, draw=0):
    """One structured answer. Cached on everything that determines it, so a rerun costs nothing.
    `draw` lets you ask the same persona the same question more than once."""
    global spent
    raw = f"{model}|{temperature}|{draw}|{persona}|{question}"
    path = CACHE / (hashlib.sha256(raw.encode()).hexdigest() + ".json")
    if path.exists():
        return json.loads(path.read_text())
    if spent >= BUDGET_USD:
        raise RuntimeError(f"Budget of ${BUDGET_USD} reached")
    response = client.chat.completions.create(
        model=model,
        messages=[{"role": "system", "content": persona}, {"role": "user", "content": question}],
        temperature=temperature,
        response_format={"type": "json_schema", "json_schema": {"name": "vote", "strict": True}}
    )
    spent += response.usage.cost
    try:
        answer = json.loads(response.choices[0].message.content)
        answer = {"vote": answer["vote"], "confidence": answer["confidence"]}
    except (TypeError, ValueError, KeyError):
        answer = {"vote": None, "confidence": None} # no answer, or not the shape we asked for
    record = {
        "model_requested": model,
        "model_answered": response.model,
        "temperature": temperature,
        "draw": draw,
        "persona": persona,
        "question": question,
        "cost": response.usage.cost,
        **answer,
    }
    path.write_text(json.dumps(record))
    return record

```

The cache key is the model, temperature, draw, persona, and question, so a rerun costs nothing and a crash can be resumed. `model_answered` is the checkpoint that replied, which differs from `model_requested` when a slug is an alias. Put the dated slug in your paper.

5. Save this as `00-check.py` and run it with `uv run 00-check.py`. Three answers print. Run it again and the cost is zero.

```

from silicon import *
persona = "You are a woman in your thirties who lives in the Midwest of the United States and  

question = "If the election for the United States House of Representatives were held today, v  

for model in MODELS:
    print(ask(model, persona, question))
print(f"Spent ${total_spent():.4f}")

```

6. Install a coding agent. Each of these works with the OpenRouter key. Pick one.

- **OpenCode**. Install with `curl -fsSL https://opencode.ai/install | bash`. Run `opencode auth login`, pick OpenRouter, and paste the key. Start it in the project folder with `opencode`. Pick a model with `/models`. Exit with `/exit`.
- **pi**. Install with `curl -fsSL https://pi.dev/install.sh | sh`. Start it in the project folder with `pi`, type `/login`, pick OpenRouter, and paste the key. Pick a model with `/model`. Exit with `/quit`.
- **omp**. Install with `curl -fsSL https://omp.sh/install | sh`. Start it in the project folder with `omp`. It reads the key from `.env`. Pick a model with `/model`. Exit with `Ctrl+C` twice.

Pick one of the three models in `silicon.py`. Ask the agent to say hello and to say which model it is. Then make an `AGENTS.md` in the project folder that says what the project is, that scripts run with `uv run`, and that the key is in `.env` and is never to be printed or committed. OpenCode writes one with `/init`. For `pi` and `omp`, write it yourself. Commit it.

7. Before Part 1: `00-check.py` runs twice and the second run costs nothing; the agent answered; `git status` does not list `.env`.

One persona, many draws

Save this as `01-draws.py` and run it. It asks each model the question from `00-check.py` twenty times at `temperature=1.0`. That is 60 calls.

```

import collections
from silicon import *

persona = "You are a woman in your thirties who lives in the Midwest of the United States and  

question = "If the election for the United States House of Representatives were held today, v

for model in MODELS:
    votes = [ask(model, persona, question, temperature=1.0, draw=i)["vote"] for i in range(20)]
    print(model, collections.Counter(votes))

```

For each model, write down the share of the twenty draws that gave the modal answer. Then change `temperature=1.0` to `temperature=0`, run it again, and write down the three shares again. The cache keeps both sets.

Thirty personas

Save this as `02-personas.py` and run it. It draws age, gender, education, and region independently from approximate 2024 United States adult shares and writes one sentence per persona. The seed is fixed so every team gets the same thirty.

```
#### Preamble ####
# Purpose: Make thirty personas from independent marginals and save them
# Author: Your names
# Date: 7 October 2026
# Pre-requisites: uv add polars numpy

import numpy as np
import polars as pl

rng = np.random.default_rng(seed=853)
N = 30

age = rng.choice(["in your twenties", "in your thirties", "in your forties", "in your fifties",
                  "in your sixties", "in your seventies", "in your eighties", "in your nineties"],
                  size=N, p=[0.17, 0.17, 0.16, 0.16, 0.17, 0.17])
gender = rng.choice(["a woman", "a man"], size=N, p=[0.51, 0.49])
education = rng.choice(["did not finish high school", "finished high school", "have some college",
                        "graduate degree"], size=N, p=[0.09, 0.28, 0.28, 0.22, 0.13])
region = rng.choice(["the Northeast", "the Midwest", "the South", "the West"], size=N, p=[0.19, 0.21, 0.35, 0.25])

personas = pl.DataFrame({"age": age, "gender": gender, "education": education, "region": region})
personas = personas.with_columns(
    (pl.lit("You are ") + pl.col("gender") + pl.lit(" ") + pl.col("age") + pl.lit(" who lives in ")
     + pl.col("region") + pl.lit(" of the United States and ") + pl.col("education") + pl.lit(" "))
)
personas.write_parquet("personas.parquet")
print(personas["persona"][0])
```

Name one pair of these four variables that is not independent in the population, and say which way the dependence goes.

Two prompts, three models

The question is the generic ballot for the United States House election on Tuesday 3 November 2026. The neutral prompt is the persona as the system message and the question as the user message. The framed prompt adds one sentence to the start of the question: “Prices for groceries and rent have risen a lot over the past year.” Save this as `03-sample.py` and run it. It runs all thirty personas through the three models under both prompts at `temperature=1.0`. That is 180 calls, eight at a time.

```
#### Preamble ####
# Purpose: Thirty personas, three models, two prompts; save the sample
# Author: Your names
# Date: 7 October 2026
# Pre-requisites: silicon.py and personas.parquet in this folder

from concurrent.futures import ThreadPoolExecutor

import polars as pl
from silicon import *

QUESTION = (
    "The election for the United States House of Representatives is on Tuesday 3 November 2026."
    "If it were held today, would you vote for the Democratic candidate, the Republican candidate,"
    "someone else, or would you not vote? How confident are you, from 0 to 100?"
)
PROMPTS = {
    "neutral": QUESTION,
    "framed": "Prices for groceries and rent have risen a lot over the past year. " + QUESTION
}
personas = pl.read_parquet("personas.parquet")

jobs = [(m, name, row["id"], row["persona"]) for m in MODELS for name in PROMPTS for row in personas]

def run(job):
    model, name, pid, persona = job
    return {"prompt": name, "id": pid, **ask(model, persona, PROMPTS[name])}

with ThreadPoolExecutor(max_workers=8) as pool:
    records = list(pool.map(run, jobs))
```

```
df = pl.DataFrame(records).join(personas.drop("persona"), on="id")
df.write_parquet("silicon_sample.parquet")
print(df.group_by("model_requested", "prompt", "vote").len().sort("model_requested", "prompt"))
print(f"Spent ${total_spent():.4f}")
```

Check `df["vote"].null_count()`. A null is a call that returned no answer or an answer that was not the shape we asked for. Report the count by model.

Model effect and prompt effect

Save this as `04-cells.py` and run it. For each of the six model-by-prompt cells it computes the Democratic share of those who said they would vote, the share who said they would not vote, and the mean and standard deviation of **confidence**.

```
import polars as pl

df = pl.read_parquet("silicon_sample.parquet")
cells = (
    df.group_by("model_requested", "prompt")
    .agg(
        (pl.col("vote") == "Democratic").sum().alias("dem"),
        (pl.col("vote").is_in(["Democratic", "Republican", "Other"])).sum().alias("voters"),
        (pl.col("vote") == "Would not vote").mean().alias("share_not_voting"),
        pl.col("confidence").mean().alias("conf_mean"),
        pl.col("confidence").std().alias("conf_sd"),
    )
    .with_columns((pl.col("dem") / pl.col("voters")).alias("dem_share"))
    .sort("model_requested", "prompt")
)
print(cells)
```

Then compute two numbers. 1. The model effect: for each prompt, the largest Democratic share across the three models minus the smallest, averaged over the two prompts. 2. The prompt effect: for each model, the Democratic share under **framed** minus the share under **neutral**, averaged over the three models. Which is larger? The standard error of a share from thirty draws is about nine points. Say whether the difference between the two effects is more than that.

Compared with a poll

Find one generic-ballot poll from the past month. The Wikipedia page on opinion polling for the 2026 United States House elections lists them with sources. Write down the pollster, the field dates, the sample size, and the Democratic and Republican shares, and cite it. Which of the six cells is closest to it? Turnout in a midterm is around half of adults. Compare that with `share_not_voting` for each model.

Differences within a group

Heath and Alexander (2026) say that the second way a silicon sample fails is that the answers within a group are too alike, even when the marginal is right. Do two checks. 1. Within each cell, the share of the thirty personas that gave the modal answer, compared with the share from one persona asked twenty times in Part 1. 2. The Democratic share by `education` within one model and one prompt, compared with the gap of ten to twenty points between the lowest and highest education groups in polls.

Write it down

Make `findings.md` in your team repo with the six-cell table, the model effect and the prompt effect, the poll with its citation, and three sentences: which effect was larger, whether the spread in Part 6 is closer to Part 1 or to a poll, and one change you would make to the persona prompt before Assignment II.

Commit the scripts, `personas.parquet`, `silicon_sample.parquet`, the cache folder, `AGENTS.md`, and `findings.md`. Do not commit `.env`. Check `git status` before you push.

References

Heath, Oscar, and Rohan Alexander. 2026. *Correcting Mode Collapse in Silicon Sampling with Semantic Similarity Rating*. arXiv. <https://doi.org/10.48550/arXiv.2607.28550>.